

PrepNova

PrepNova — RAG-based adaptive mock interview platform

Student ID	Name
202512001	Shruti Makwana
202512015	Jaimin Trivedi
202512023	Kush Dani
202512064	Krishha Shah
202512065	Dev Patel
202512067	Varunee Agrawal
202512085	Hardi Dhadhal
202512097	Garv Modi

Project Description

PrepNova is an AI-powered web application designed to help students, fresh graduates, and professionals prepare for job interviews through personalized mock interview sessions and resume-aware question generation. The platform leverages Retrieval-Augmented Generation (RAG) to create interview experiences tailored to each user's profile.

Users can upload their resume or manually select a target job role to begin an AI-powered mock interview. Based on the uploaded resume or selected role, the system retrieves relevant context and generates customized technical and HR interview questions across three difficulty levels. After the interview, PrepNova provides a performance report including an AI-based evaluation score, identified strengths and weaknesses, and personalized improvement suggestions.

Application Type

PrepNova is a web application, accessible through a browser without any installation. It follows a client-server architecture, where the frontend interacts with the backend through RESTful APIs. The platform combines a web-based interface with an AI/RAG-powered backend to deliver personalized, resume-aware interview preparation.

Backend REST API Modules

Module	Representative Endpoints
Auth	POST /api/auth/register, POST /api/auth/login, GET /api/auth/me
Resume & Role	POST /api/resume/upload, POST /api/role/select
Interview Session	POST /api/interview/start, GET /api/interview/:id/next-question, POST /api/interview/:id/answer
Evaluation & Reports	GET /api/interview/:id/report, GET /api/history
RAG Service (internal)	POST /rag/parse, POST /rag/embed, POST /rag/retrieve, POST /rag/generate-question — called by the Node backend, not exposed directly to the frontend

Core Database Collections (MongoDB)

Collection	Key Fields
users	_id, name, email, passwordHash, createdAt
resumes	_id, userId, rawText, chunks[] (text, sectionLabel, embeddingId), uploadedAt
interviewSessions	_id, userId, role, difficulty, questions[] (questionId, text, type, difficulty, source), startedAt, completedAt
answers	_id, sessionId, questionId, answerText, score, strengths[], weaknesses[], submittedAt
questionBank	_id, role, difficulty, type, questionText — curated fallback questions used when no resume is uploaded

Frontend Pages

- Login / Register page
- Resume Upload / Role Selection page
- Difficulty Selection page
- Interview Session page — question display, answer input, progress indicator
- Report / Score page — per-session score, strengths, weaknesses
- Dashboard / Interview History page — past sessions and score trend over time

Motivation / Problem Statement

Preparing for job interviews is challenging because most candidates lack access to personalized interview practice and detailed feedback. Existing platforms rely on static question banks or generic chatbots that cannot understand a candidate's resume, technical skills, or career goals.

PrepNova addresses this by combining AI-powered mock interviews with resume-aware question generation. Using RAG, the platform retrieves relevant context from a candidate's resume (or a curated role-based knowledge base when no resume is uploaded) to generate targeted questions, evaluate responses, and provide actionable feedback on interview performance.

Target Users and Use Cases

Target Users

College Students, Fresh Graduates, Working Professionals, Internship Applicants, Job Seekers

Use Cases

- Register and securely log in to the platform.
- Upload a resume, which is parsed and embedded for RAG-based question generation.
- Select a target job role if a resume is not uploaded (questions retrieved from a curated role-based question/knowledge base).
- Choose an interview difficulty level (Easy, Medium, or Hard).
- Participate in AI-generated technical and HR mock interviews.
- Receive a post-interview evaluation report with score and improvement areas.
- Access previous interview reports to monitor progress over time.

Scope and High-Level Features

- Resume upload with PDF parsing and text extraction
 - Text is extracted using pdfplumber for PDFs (preserves section ordering on multi-column layouts better than PyPDF2) and python-docx for Word files, then cleaned: whitespace normalized, repeated headers/footers stripped.
- RAG pipeline: resume/JD embedding → vector store retrieval → context-grounded question generation
 - Chunking: cleaned text is split with a recursive character/token splitter, target chunk size ~300 tokens with 50-token overlap, each chunk tagged by resume section (Education/Experience/Projects/Skills).

- Embedding: each chunk is converted to a vector using sentence-transformers/all-MiniLM-L6-v2 (384-dim, runs locally at no per-call cost); a hosted embedding API is the fallback if retrieval quality proves insufficient during testing.
- Vector store: FAISS (IndexFlatL2) for the prototype, with embeddings namespaced per user_id/session_id so retrieval never crosses users; ChromaDB is the evaluated fallback if on-disk persistence across server restarts is needed.
- Retrieval: a query embedding built from the target role/skill and difficulty level pulls the top-k (k = 3–5) most similar chunks via cosine similarity, restricted to that user's namespace.
- Generation: retrieved chunks are inserted into a fixed prompt template with explicit instructions (target role/skill, question type, difficulty); the LLM response is parsed to keep only the question text before storage.
- Curated role-based knowledge base for users who select a role instead of uploading a resume
 - When no resume is uploaded, the RAG retrieval step is skipped entirely; questions are pulled directly from a pre-tagged question bank in MongoDB, filtered by role and difficulty, optionally passed through the LLM once for light rephrasing so repeat users don't see identical wording.
- Manual job role selection dropdown
- Three interview difficulty levels (Easy, Medium, Hard)
 - Difficulty is enforced through explicit rubric language in the generation prompt: Easy = definition/recall question, Medium = applied scenario question, Hard = trade-off or design-judgment question.
- Dynamic technical and HR interview question generation
 - Each session generates a fixed set of questions up front (5 technical + 2 HR) at the user-selected difficulty, rather than adapting difficulty turn-by-turn based on the previous answer — real-time adaptive difficulty is listed under Future Scope, not built in this version, to keep the system deterministic and testable.
 - HR/behavioural questions are always drawn from the curated question bank (not RAG), since they are role-generic rather than resume-specific.
- Post-interview AI-based evaluation and scoring (out of 10)
 - The evaluation prompt includes the original question, the candidate's answer, and an explicit rubric: Correctness (0–4), Completeness (0–3), Clarity (0–3).
 - The LLM is instructed to return a fixed JSON structure ({ score, strengths, weaknesses }) rather than free text, so the backend can parse it reliably; if parsing fails, a default fallback response is stored instead of the session crashing.
- Personalized improvement feedback (strengths/weaknesses)
 - Strengths and weaknesses are generated per-question by the evaluation LLM call, then aggregated at the session level: recurring weakness themes (e.g., "lacked depth on X" across 2+ questions) are surfaced first on the report page, ahead of one-off comments.
- Interview history and performance tracking
 - Every completed session is saved with its full question set, answers, per-question scores, and the session's overall score; the dashboard queries this by userId and renders a simple score-over-time view (session date on x-axis, score out of 10 on y-axis) so a user can see whether they're improving across attempts.
- Secure user authentication using JWT
 - Passwords hashed with bcrypt before storage; JWT access token issued on login (short expiry) and attached as a Bearer token on all protected routes; middleware on the Express side rejects requests with missing/expired/invalid tokens before they reach any controller logic.
- Basic error handling (unsupported file formats, empty/corrupt resumes, parsing failures, malformed AI evaluation output)
 - File upload is validated on both size and MIME type before being sent to the parsing service; a corrupt or unparseable file returns a specific error message to the frontend (not a generic 500) so the user knows to re-upload rather than retry blindly.
 - If resume parsing yields under a minimum text-length threshold (e.g., a scanned/image-only PDF with no extractable text), the user is prompted to select a role instead, falling back to the curated question bank path.

- Automatic job role detection from resume, with lite ATS compatibility scoring
 - Keyword/skill extraction from the parsed resume text suggests a likely target role automatically when the user has not manually selected one, reusing the same parsing pipeline built for RAG.
 - ATS-lite scoring compares the resume's extracted keywords against the expected skill list for the selected/detected role and returns a weighted match score, shown alongside the interview report — not a full formatting/structure ATS audit, but a directional keyword-match signal.
- Real-time adaptive difficulty within a session
 - Each subsequent question's difficulty adjusts based on the score of the previous answer (score $\geq 8 \rightarrow$ increase difficulty, $\leq 4 \rightarrow$ decrease), rather than staying fixed for the whole session as in a static-difficulty design.
- CI/CD pipeline and security hardening
 - GitHub Actions runs the unit test suites (Jest/Supertest for backend, Pytest for the AI/RAG service) on every pull request, with an automated staging deployment step on merge to main.
 - Rate limiting (express-rate-limit) on authentication and resume-upload endpoints, input sanitization audit across all POST endpoints, and a review of prompt-injection risk in the RAG prompt construction.
- Automated end-to-end testing and performance/load testing
 - Cypress E2E tests cover the two core flows (resume-based and role-based) end-to-end; Locust/k6 load testing is run against the interview-session, evaluation, and retrieval endpoints to document latency/throughput and surface bottlenecks.
- Accessibility and documentation
 - A manual accessibility pass (alt text, colour contrast, keyboard navigation) across all pages, alongside code documentation (JSDoc/docstrings) and a maintained API reference (Postman collection / OpenAPI spec).

Technology Stack

Layer	Technology
Frontend	React.js, Tailwind CSS, JavaScript
Backend	Node.js, Express.js
AI / RAG Layer	Python (Flask/FastAPI service — RAG pipeline, embeddings, and model integration)
Database	MongoDB
Vector Store	FAISS (prototype) — ChromaDB evaluated as the persistence fallback
Authentication	JWT Authentication
AI Model	Open-source LLM (specific model yet to be finalized)
AI Framework	Retrieval-Augmented Generation (RAG)
Resume Processing	pdfplumber / python-docx (parsing), recursive chunking, sentence-transformers embeddings
Testing Tools	Jest + Supertest (Node/API), React Testing Library (UI), Pytest (Python AI/RAG service), Cypress (E2E), Locust/k6 (load testing)

Layer	Technology
CI/CD	GitHub Actions (automated test runs on PRs, staging deployment on merge)
Version Control	Git & GitHub

Roles and Responsibilities

Team Member	Responsibility
Varunee	Backend: Authentication (JWT), server integration, API security
Hardi	Frontend: UI/UX design, resume upload interface, report visualization
Krishna	AI/RAG: Resume/JD parsing, chunking, and embedding pipeline
Shruti	Frontend: Interview interface + dashboard (question flow, difficulty selection, history UI)
Kush	Backend: REST API development and database schema design (MongoDB)
Jaimin	AI: Evaluation/scoring system, deployment, testing coordination
Dev	AI: Question generation logic (prompt engineering, difficulty-level tuning)
Garv	AI/RAG: Vector store integration and retrieval logic

Timeline with Milestones

The project runs across a 14-week schedule — an 8-week core-platform build followed by a 6-week extended-features, hardening, and testing phase. Testing runs alongside development throughout, rather than as a separate end-of-project phase; the specific unit-test and integration-check activity for each week is detailed in the accompanying Project Plan document (Section 1), along with full per-member effort estimates.

Week	Milestone
Week 1	Requirement analysis, literature survey, project planning, UI design
Week 2	Database design, backend setup, REST API development, authentication
Week 3	Resume upload, PDF parsing, text chunking, embedding pipeline setup
Week 4	Vector store integration, RAG retrieval logic, role-based knowledge base
Week 5	Dynamic question generation across difficulty levels, mock interview flow
Week 6	AI-based evaluation, scoring, and feedback report generation
Week 7	Dashboard, interview history, integration testing, bug fixing
Week 8	MVP stabilization — regression fixes, internal checkpoint demo
Week 9	Automatic role detection + lite ATS compatibility scoring module

Week	Milestone
Week 10	Real-time adaptive difficulty + curated question bank expansion
Week 11	CI/CD pipeline setup + security hardening
Week 12	Automated E2E test suite (Cypress) + performance/load testing
Week 13	Accessibility pass, UI polish, code documentation
Week 14	Final regression testing, production deployment, documentation, presentation prep

Expected Outcome

PrepNova aims to provide a realistic, resume-aware interview preparation experience by integrating a RAG-based question generation pipeline with AI-driven evaluation, backed by CI/CD-enforced testing, security hardening, and accessibility review. The platform will help users practice with context-relevant questions, receive structured feedback on their performance, and track improvement over multiple sessions.

Future Scope

- Subject-wise learning recommendations and career roadmap generation
- Voice-based AI interviewer (Speech-to-Text / Text-to-Speech)
- Online coding interview environment
- Company-specific interview preparation (Google, Amazon, Microsoft, etc.)
- Google Meet integration for mentor-led interviews
- Calendar scheduling and reminder notifications
- Full ATS audit (formatting/structure compliance), beyond the keyword-match scoring built in Week 9